

SECTION 25 — API TESTING AND SYSTEM TESTING SCENARIOS (Refined)

Topics Covered

- 25.1-25.15 (15 headers)

First pass — 2 parallel batch subagents

25.1 End-to-End API Flow — Refined

Theory: End-to-end API flow validates a complete business journey across multiple endpoints in sequence — not isolated calls. Typical pattern: `POST /login → POST /orders → GET /orders/{id} → PATCH → DELETE`, chaining dynamic IDs and tokens between steps. Assert status, schema, DB state, and side effects at each hop (not just final). In RestAssured/Playwright, store `orderId` from one response for the next request. Isolate data with setup/teardown fixtures and unique test payloads for CI repeatability.

Enterprise Relevance: Single-endpoint tests miss integration breaks (auth → order linkage, stale IDs, cross-service transactions). Deloitte E2E suites chain 5-10 endpoints per journey with per-hop asserts — a failure pinpoints the exact hop.

```
const create = await request.post('/orders', { data: order });
expect(create.status()).toBe(201);
const id = (await create.json()).id;
const get = await request.get(`/orders/${id}`);
await expect(get).toBeOK();
await expect((await get.json()).status).toBe('CREATED');
```

High-Stakes Scenario: Each endpoint green alone, journey red (order created but payment link missing `orderId` propagation). Triage: hop-by-hop asserts + chained IDs + DB verification per hop; contract tests per endpoint + E2E for linkage.

Anti-Patterns: Asserting only final hop; hardcoded IDs across hops; no cleanup (chained data accumulates); single-endpoint tests labeled "E2E".

Qs: What does each hop assert? How do you isolate chained journeys in parallel?

25.2 Positive / Negative Testing — Refined

Theory: Positive testing verifies happy-path behavior with valid, complete payloads: expect `200/201`, correct schema, persisted state. Negative testing proves robustness with invalid inputs: missing fields, wrong types, malformed JSON, duplicate creates, non-existent IDs — expecting `400/404/422` with clear errors (never 500 for client errors). Map every required/optional field to valid, invalid, and null cases. Parameterize with `@CsvSource / test.each` to cover combinations without duplication; assert both code and message.

Enterprise Relevance: Negative coverage is where security and data-corruption bugs hide. Deloitte field-matrix standard: each field × {valid, missing, wrong-type, null, boundary} — generated, not handwritten.

```
test.each([ { p: {} }, { p: { age: "xx" } }, { p: { age: -1 } } ])( 'reject %j', async ({ p }) => {
  const r = await request.post('/users', { data: p });
  expect(r.status()).toBe(400);
  expect((await r.json()).code).toMatch(/VALIDATION_/);
});
```

High-Stakes Scenario: Missing-field request returns 500 + stack trace (info leak + false 5xx alarming). Triage: negative matrix per endpoint, assert exact 4xx + error contract (Sec 25.7), no-stack-trace check, required-field checklist from OpenAPI.

Anti-Patterns: Happy-path only; asserting status without message; 500 for validation errors; handwritten combos (use parameterized).

Qs: How do you generate negative matrices from OpenAPI? What must a negative assert beyond status?

25.3 Boundary Conditions — Refined

Theory: Boundary testing targets edges where off-by-one defects hide: min-1, min, max, max+1 for numbers, string lengths, arrays, pagination, dates. If `quantity: 1-100` or `name ≤ 50 chars`, test `0`, `1`, `100`, `101` and `49`, `50`, `51` chars, plus empty, null, unicode. Combine with equivalence partitioning to reduce cases. Automate limits from OpenAPI `minimum/maximum/maxLength`; assert `400` outside range vs `200/201` on edge; check DB for silent truncation.

Enterprise Relevance: Boundaries are the highest-yield negative tests per case (5 cases catch what 50 random values miss). Generate from OpenAPI constraints — never hand-maintain boundary lists.

```
@pytest.mark.parametrize("qty,code", [(0,400),(1,201),(100,201),(101,400)])
def test_qty(qty, code): assert post_order(qty).status_code == code
# string: 49/50/51 chars, empty, 10k chars (DoS guard), unicode emoji
```

High-Stakes Scenario: `VARCHAR(50)` truncates 60-char names silently — UI shows full, DB stores partial, downstream matching breaks. Triage: boundary asserts on response + DB length, OpenAPI `maxLength` enforcement test, truncation-vs-rejection contract decision documented.

Anti-Patterns: Only happy-mid values; manual boundary lists (drift from spec); no DB-length check; missing empty/null/unicode.

Qs: min-1/min/max/max+1 for which types? How do you derive boundaries from OpenAPI?

25.4 Validation of Business Rules — Refined

Theory: Business rules go beyond syntax to enforce domain invariants: insufficient balance blocks transfer, expired coupon rejected, overlapping booking conflict, status transitions only `PENDING→APPROVED` (never `PENDING→SHIPPED`). Model preconditions via API setup, execute action, then verify `422/409` + DB/ledger effects. Keep rules data-driven in a decision table for auditability. Negative rule tests are high-value for interviews — trace requirements to test IDs, verify error `code` maps to the specific violated rule.

Enterprise Relevance: Rule bugs are money bugs (double-spend, invalid state transitions). Decision-table coverage is auditable proof for regulated clients.

```
await deposit(account, 50);
const r = await request.post('/transfer', { data: { amount: 100 } });
```

```
expect(r.status()).toBe(422); // INSUFFICIENT_FUNDS
expect((await r.json()).code).toBe('INSUFFICIENT_FUNDS');
// state machine: PENDING→APPROVED ok, PENDING→SHIPPED → 409 INVALID_TRANSITION
```

High-Stakes Scenario: Status skip (PENDING → DELIVERED directly) corrupts fulfillment pipeline — rule enforced in UI only, API accepts anything. Triage: state-transition matrix test (all pairs valid/invalid), server-side enforcement (never UI-only), DB constraint backup.

Anti-Patterns: Syntax-only validation (no rules); UI-only rule enforcement; generic 400 without rule code; untested transition matrix.

Qs: How do you test a state machine exhaustively? Why must rules live server-side?

25.5 Authentication Failures — Refined

Theory: Authentication answers who you are; failures must return **401 Unauthorized** — never **403** (that's authorization) or data leakage. Cover: missing **Authorization** header, malformed **Bearer xxx**, invalid signature, expired token, wrong issuer/audience, revoked refresh token. Verify error contract (`{ "error": "invalid_token" }`), no stack traces. Automate with short-lived tokens / clock skew; test token refresh rotation. Assert all protected endpoints (including internal routes exposed externally) enforce auth uniformly.

Enterprise Relevance: Auth gaps are breach vectors. Every endpoint needs an auth test; unprotected internal routes exposed via gateway misconfiguration are a classic finding.

```
expect((await request.get('/orders')).status()).toBe(401); // no token
expect((await request.get('/orders', { headers: { Authorization: 'Bearer bad' } })).status()).to
// expiry: mint 1s token, wait, assert 401 + WWW-Authenticate: Bearer
```

High-Stakes Scenario: Health-adjacent `/internal/stats` exposed without auth (gateway wildcard) — data leak.

Triage: auth matrix test (every endpoint × no/invalid/expired token), gateway config audit, **WWW-Authenticate** header check, no-stack-trace assertion.

Anti-Patterns: **403** for auth failures (wrong code); stack traces on **401**; testing only happy auth; internal routes untested.

Qs: **401** vs **403** — who decides which? What must a **401** response contain and never contain?

25.6 Authorization Failures — Refined

Theory: Authorization answers what you may do after valid authentication; violations must return **403 Forbidden**.

Test: horizontal access (user A reads user B's `/orders/{id}`), vertical escalation (user calls `/admin/users`), role matrix (admin/editor/viewer × CRUD), IDOR via guessed UUIDs. Verify masked **404** vs **403** per design (**404** prevents ID enumeration; **403** confirms existence — design decision with security tradeoff). Automate with two-role tokens, cross-tenant calls, assert resources unchanged. Log access-denied audits where compliance requires.

Enterprise Relevance: IDOR/BOLA is OWASP API #1 — authorization tests are security tests. Every object-level endpoint needs horizontal + vertical negative tests.

```
const userToken = await login('user');
const r = await request.delete('/admin/users/1', { headers: { Authorization: userToken } });
expect(r.status()).toBe(403);
// horizontal: userA GET userB/order → 403 (or 404 by design) + resource unchanged
```

High-Stakes Scenario: Guessed UUID accesses another tenant's invoices (missing object-level check; only route-level auth). Triage: IDOR matrix (every ID endpoint × foreign ID), 404-vs-403 design review, audit-log verification, tenant-scoping at query layer (not controller).

Anti-Patterns: Route auth without object checks; 500 on forbidden (leaks); no cross-tenant tests; relying on unguessable UUIDs as security.

Qs: Horizontal vs vertical vs IDOR? 403 vs 404 masking trade-off?

25.7 Error Contract Validation — Refined

Theory: Error contract ensures every failure returns a predictable, versioned shape clients can handle: RFC 7807

`ProblemDetails {type, title, status, detail, traceId, errors[]}`. Validate JSON schema, required fields, `status` mirroring HTTP code, stable machine-readable `code`, no PII/stack leakage, `Content-Type: application/problem+json`. Automate with AJV/schema validation across 400/401/403/404/422/500 + snapshot `traceId` correlation across logs. Distinguish retryable (503/429) from fatal (400) by contract fields — clients branch on them.

Enterprise Relevance: Inconsistent errors break client handling and mask retryability. Contract tests on errors are as important as happy-path schema tests; Deloitte API packs assert error shape for every negative case.

```
expect(r.status()).toBe(422);
expect(await r.json()).toMatchObject({ status: 422, code: expect.any(String), traceId: expect.any(String) });
expect(r.headers()['content-type']).toContain('application/problem+json');
// no stack: expect(JSON.stringify(body)).not.toContain('at com.');
```

High-Stakes Scenario: 500s return HTML stack traces with DB credentials in production logs + client-visible. Triage: error-contract schema test per status family, PII/stack scan on error bodies, `traceId` log correlation verified end-to-end.

Anti-Patterns: Bare string errors; status/code mismatch; stack traces in responses; changing error shape without versioning; no `traceId`.

Qs: What must every error response contain? How do clients distinguish retryable from fatal?

25.8 Rate Limiting — Refined

Theory: Rate limiting protects APIs from abuse via 429 Too Many Requests + Retry-After, X-RateLimit-Limit/Remaining/Reset headers. Test: burst vs sustained limits, per-key (IP/token), endpoint-specific quotas, limit reset after window. Automate by looping requests until 429, asserting backoff compliance and eventual recovery; mock clock where possible to avoid slow tests. Verify priority lanes (authenticated higher quota than anonymous) and that 503 circuit-breaker is distinct from quota 429 for correct client retry logic.

Enterprise Relevance: Rate limits are a contract — clients implement backoff against documented headers. Untested limits mean launch-day throttling surprises + retry storms that amplify outages.

```
for _ in range(60):
    r = get("/search", headers=auth)
    if r.status_code == 429:
        assert "Retry-After" in r.headers
        assert int(r.headers["X-RateLimit-Remaining"]) == 0
        break
    else: raise AssertionError("never rate-limited")
```

High-Stakes Scenario: Parallel suite (16 workers) trips rate limit → 200 false 429 failures blamed on product. Triage: test-double or raised test-tier quota, per-worker API keys (isolated buckets), backoff+jitter in client, 429-vs-503 distinction in triage guide.

Anti-Patterns: No rate-limit tests; hammering prod limits from CI; retrying 429 immediately (no backoff); confusing 429 with 503.

Qs: How do you test limits without slow loops? 429 vs 503 — different client behavior?

25.9 Retry Behavior — Refined

Theory: Retry handling ensures transient failures (timeouts, 502/503/504, 429s) recover without user impact.

Production uses exponential backoff with jitter, max attempts, and timeouts to avoid thundering herd. SDET must verify: only idempotent calls retry, non-transient 4xx never retry, budgets exhaust correctly (no infinite loops). Test with fault-injecting mocks: fail twice then succeed; assert attempt count, delay windows, and no duplicate side effects.

Enterprise Relevance: Retry policy is a reliability contract — wrong retries duplicate charges (POST retried without key) or hide outages (infinite retry on 500). Every client retry config needs a test.

```
# fail first 2 calls, assert client retries 3x with backoff then succeeds
responses = [503, 503, 200]
assert client.post("/pay", retries=3).status_code == 200
assert mock.call_count == 3
# assert delays: ~100ms, ~200ms (exp backoff + jitter bounds)
```

High-Stakes Scenario: Client retries 500s infinitely — outage becomes retry storm, doubling load and extending downtime. Triage: max attempts (3) + backoff + jitter + circuit breaker; retry only idempotent/keyed; 4xx never retried; budget metrics (retry rate alert).

Anti-Patterns: Retrying non-idempotent POST without key; infinite retries; no jitter (thundering herd); retrying 400/422 (waste, masks bugs).

Qs: Which failures are retryable? How do backoff + jitter + circuit breaker interact?

25.10 Idempotency — Refined

Theory: Idempotency means N identical requests equal one execution. GET, PUT, DELETE, HEAD, OPTIONS are naturally idempotent; POST/PATCH are not unless designed with `Idempotency-Key`. Server caches response by key and replays it without re-executing. SDET must test: duplicate POSTs with same key (single resource, same ID, replay flagged), different keys (two resources), no key (two resources). Verify no double billing.

Enterprise Relevance: Payment/order double-submit is the money bug idempotency prevents. Every mutating POST needs an idempotency test — same key twice, different keys, missing key.

```
h = {"Idempotency-Key": "uuid-abc"}
r1 = post("/orders", json=order, headers=h) # 201
r2 = post("/orders", json=order, headers=h) # 200 replay (or 201 same ID)
assert r1.json()["id"] == r2.json()["id"]
assert count_orders() == 1
# different key → second resource; no key → two resources
```

High-Stakes Scenario: Double-click checkout charges twice (\$180 instead of \$90) — no idempotency key. Triage: key on all mutating POSTs (client-generated UUID), server key store with TTL, replay detection tests, double-click E2E

test.

Anti-Patterns: Retrying POST without key; keys not persisted (restart loses dedup); same key different payload (must reject 422, not replay); no TTL (key store grows forever).

Qs: Which methods are naturally idempotent? What must the server do on same-key-different-payload?

25.11 Eventual Consistency — Refined

Theory: Eventual consistency guarantees replicas converge if writes stop — but reads may be stale during the inconsistency window. Common in Elasticsearch, Cassandra, S3, async replication with $W+R \leq N$ quorum tuning. SDET cannot assert immediately; use AAAA pattern: Arrange, Act, Await convergence, Assert. Poll with timeout instead of fixed sleep; verify monotonic reads and read-your-writes where promised.

Enterprise Relevance: Search/indexing assertions without convergence waits are the top async flake source. Every eventually-consistent read needs a polling wrapper with timeout, not a sleep.

```
# poll until indexed – never sleep fixed time
create_doc(id="1")
assert poll(lambda: search("id:1"), timeout=10) is not None
# AAAA: arrange doc, act index, await visible, assert content
```

High-Stakes Scenario: "Created order not in search" test fails 20% (index lag 1-8s). Triage: Await-convergence polling (timeout 15s, interval 500ms), read-your-writes session consistency where offered, lag metric dashboard to set realistic timeouts.

Anti-Patterns: Fixed sleeps for convergence; asserting immediate global visibility; no timeout (hangs forever); confusing eventual with broken.

Qs: AAAA pattern steps? How do you test convergence without flaky sleeps?

25.12 Asynchronous Systems — Refined

Theory: Asynchronous systems decouple request from processing via callbacks, promises, schedulers, and events — creating nondeterminism, race conditions, and flaky tests. Microsoft studies show Async Wait is the leading flake cause. SDET strategy: synchronize on completion signals (webhooks, status endpoints, events), not sleeps; stub dependencies; use Awaitility/ `expect.poll`; isolate unit logic synchronously. Verify ordering, timeouts, callbacks never lost, failures surface observably (not silent drops).

Enterprise Relevance: Async flows (order → payment webhook → fulfillment) are E2E-critical and E2E-flaky.

Completion-signal synchronization + timeout + dead-letter assertions form the async testing contract.

```
# await async side-effect via signal, not sleep
await sut.start_async()
await asyncio.wait_for(sut.done_event.wait(), timeout=5)
assert sut.message == "Init Work"
# API equivalent: poll status endpoint until terminal state
```

High-Stakes Scenario: Webhook never arrives (lost, no retry) — order stuck PENDING, test times out with no diagnosis. Triage: completion-signal wait + timeout, DLQ assertion (failed messages land in DLQ, none lost silently), idempotent handlers (duplicate delivery safe), observability (trace ID across async hops).

Anti-Patterns: Fixed sleeps for async; no timeout (hangs); asserting intermediate states as final; untested DLQ; fire-

and-forget without delivery guarantees.

Qs: How do you test async completion deterministically? What proves no message is silently lost?

25.13 Message Queues — Awareness — Refined

Theory: Queues (Kafka, RabbitMQ, SQS) decouple producers/consumers for buffering, retries, and scaling. Kafka: topics, partitions, offsets, consumer-groups with replay; ordering per partition; at-least-once delivery (duplicates possible). RabbitMQ: exchanges, routing-keys, queues with push delivery. Both use acknowledgments (no ack = redeliver) and Dead Letter Queues for poison messages (failing after N retries). SDET awareness: verify ordering per partition/queue, at-least-once duplicates handled idempotently, DLQ routing, no loss on broker/consumer failure. Enterprise Relevance: Queue semantics dictate test design: at-least-once → idempotent consumers mandatory; DLQ → poison-message tests; partitioning → ordering tests per key.

```
# poison message lands in DLQ after retries (not lost, not blocking)
produce("orders", bad_msg)
assert poll(lambda: count("orders.DLQ") == 1, timeout=15)
# ordering: same key → same partition → ordered; assert sequence per key
```

High-Stakes Scenario: Poison message without DLQ blocks partition forever — all subsequent orders stall. Triage: DLQ configured + tested (poison → DLQ in N retries), consumer lag alerts, idempotent handlers (duplicate delivery safe), ordering assertions per partition key.

Anti-Patterns: Assuming exactly-once (Kafka/RabbitMQ don't guarantee); no DLQ test; global ordering assumption (only per-partition); untested consumer-crash recovery (offset loss).

Qs: At-least-once vs exactly-once — what must consumers handle? What happens to poison messages?

25.14 Database Consistency — Refined

Theory: Database consistency spans CAP and PACELC tradeoffs: during a Partition choose Availability vs Consistency; Else (normal operation) choose Latency vs Consistency. Models range: eventual < session < causal < serializable/linearizable (strongest). SDET verifies: constraints, isolation levels, replica convergence, stale reads, conflict resolution. Check redundant replicas return same data after propagation, failed node rebuild loses nothing, transactions preserve unique keys and invariants.

Enterprise Relevance: Consistency-model awareness prevents false failures (asserting immediate global visibility on an eventually-consistent store) and catches real anomalies (lost writes, split-brain divergence).

```
# write to leader, verify all replicas converge (with timeout, not sleep)
write(node1, {"id": 1, "bal": 100})
assert poll(lambda: all(read(n, 1) == 100 for n in replicas), timeout=10)
# conflict: concurrent writes → last-writer-wins or CRDT merge? assert documented rule
```

High-Stakes Scenario: Multi-region active-active: concurrent balance updates lose one write (last-writer-wins silently). Triage: consistency model documented per store, concurrent-write tests asserting merge rule, conflict metrics dashboard, linearizable operations for money movements.

Anti-Patterns: Assuming strong consistency everywhere; no replica-lag tolerance in tests; untested failover (node loss = data loss discovered in prod).

Qs: CAP vs PACELC? Which consistency level does your primary store offer, and what test does that require?

25.15 Distributed System Testing — Interview Depth — Refined

Theory: Deloitte SDET interviews expect systems thinking beyond scripts: retries, idempotency, consistency, queues, CAP, observability, and testability hooks — answered as scenario + mechanism + verification. Common prompts: test async workflow (completion signals + DLQ), handle flake 1/10 runs (quarantine + classify + fix), design order pipeline with DLQ (poison routing + idempotent handlers), prove no data loss on node failure (replica convergence + failover test). Show polling helpers, fault injection, contract + integration + chaos layers, and metrics (flake-rate <1%).

Enterprise Relevance: This is the Senior differentiator — connecting 25.1-25.14 into architecture judgment. Every answer: mechanism first, then how you'd verify it, then the metric proving it.

```
# interview-ready async assertion pattern (show this on whiteboard)
def await_condition(fn, timeout=10, interval=0.5):
    end = time.time() + timeout
    while time.time() < end:
        if fn(): return True
        time.sleep(interval)
    raise AssertionError("condition not met within timeout")
```

High-Stakes Scenario: "Design testing for our order pipeline" (open-ended). Triage answer: contract tests per service boundary + E2E happy path + idempotency tests + DLQ tests + chaos (kill broker/consumer) + replica-convergence checks + observability (trace IDs) + flake dashboard. Layered, not just E2E.

Anti-Patterns: Tool-only answers (no consistency/retry reasoning); "test everything E2E"; no fault injection; no metrics; confusing 401/403/429/503 handling.

Qs: Design order-pipeline testing in 5min? How do you prove no loss on node failure?